Thread in C

Primitive

```
int pthread_create (
        phtread_t* thread,
        const pthread_attr_t* attr,
        void* (*start_routine)(void*),
        void* arg);
```

- **thread**: puntatore a variabile di tipo **pthread_t,** su cui verrà memorizzato l'ID del thread creato;
- **attr**: attributi di creazione (sempre NULL in questo corso);
- **start_routine**: funzione da eseguire (prende sempre come argomento un void* e restituisce un void*);
- **arg**: puntatore da passare come argomento alla funzione start_routine;
- Ritorna 0 in caso di successo, altrimenti la causa dell'errore.

```
void pthread_exit (void* value_ptr);
```

- Termina il **thread corrente**, rendendo disponibile il valore puntato da **value_ptr** ad una eventuale operazione di join;
- All'interno di **start_routine**, può essere sostituita da **return**.

```
int pthread_join(phtread_t thread, void** value_ptr);
```

- Attende esplicitamente la terminazione del thread con ID **thread**;
- Se **value_ptr != NULL**, vi memorizza il valore eventualmente restituito dal thread (un **void***, tramite **pthread_exit**);
- Ritorna il valore 0 in caso di successo, altrimenti la causa dell'errore.

```
int pthread_detach(phtread_t thread);
```

- Notifica al sistema che non ci saranno operazioni di join su thread;
- Ritorna il valore 0 in caso di successo, altrimenti la causa dell'errore.

Esempio 1.

```
#include <stdio.h>
#include <errno.h>
#include <pthread.h>

void* thread_stuff(void *arg) {
    // codice thread che non usa argomenti
    return NULL;
}

int ret;
pthread_t thread;
ret = pthread_create(&thread,NULL,thread_stuff,NULL);
if (ret != 0) {
    fprintf(stderr, "ERROR with pthread_create!\n");
    exit(EXIT_FAILURE);
}
// codice main indipendente dal thread
ret = pthread_join(thread, NULL);
if (ret != 0) [...]
```

Esempio 2. Thread Multipli

```
#include <stdio.h>
#include <errno.h>
#include <pthread.h>
#define NUM_THREADS 4


void *hello(void *arg) {
      printf («Hello Thread\n»);
}

main() {
  pthread_t tid [NUM_THREADS];
  for (int i = 0; i < NUM_THREADS; i++)
    ret = pthread_create(&tid[i], NULL, hello, NULL);

  for (int i = 0; i < NUM_THREADS; i++)
    ret = pthread_join(tid[i], NULL);
}
```

Esempio 3. Passaggio di Argomenti (lancio di N thread con argomenti necessariamente distinti)

```
#include <errno.h>
#include <pthread.h>

void* foo(void *arg) {
  type_t* obj_ptr = (type_t*) arg; // cast ptr argomenti
  /* <corpo del thread> */
  free(obj_ptr);
  return NULL;
}

int ret;
pthread_t* threads = malloc(N * sizeof(pthread_t));
type_t* objs = malloc(N * sizeof(type_t));
for (i=0; i<N; i++) {
  objs[i] = [...] // imposto argomenti thread i-esimo
  ret = pthread_create(&threads[i], NULL, foo, &objs[i]);
  if (ret != 0) [...]
}
```

Processi

Processi vs Thread

- **Performance**: lanciare/terminare un thread è più veloce rispetto a lanciare/terminare un processo;
- **Memoria**: la creazione di un processo via fork() comporta la copia dell'intera memoria del padre, mentre i thread la condividono;
- **Comunicazione**: i thread di uno stesso processo possono usare la sua memoria per comunicare tra loro, mentre la comunicazione tra processi richiede meccanismi aggiuntivi (Inter-Process Communication).

Misurazione performance

Serve a misurare il **tempo di esecuzione di una porzione di codice**. In fase di compilazione bisogna:

- includere il **modulo performance.c**;

- linkare le **librerie run time** (**-lrt**) e **math** (**-lm**);

Esempio di utilizzo:

```
#include "performance.h"
…
timer t; begin(&t);
// porzione di codice di cui cronometrare l'esecuzione
end(&t); unsigned long int time;
time=get_seconds(&t); time=get_milliseconds(&t);
time=get_microseconds(&t); time=get_nanoseconds(&t);
```

DIPARTI AUTOMA

Tempi di lancio/terminazione

- **Misurazione della reattività** (tempo richiesto) di processi e thread nelle fasi di lancio e terminazione;
- Sorgenti: **reactivity-processes.c**, **reactivity-threads.c**;

Compilazione:

```
gcc -o reactivity-processes reactivity-processes.c
performance.c -lrt -lm
gcc -o reactivity-threads reactivity-threads.c
performance.c -lpthread -lrt -lm
```

In ogni programma vengono eseguiti **N test** per calcolare le r**eattività medie**: N è passato come argomento dalla linea di comando.

Lo **speedup** deve essere calcolato come: $\text{speedup} = \frac{\textit{tempo reactivity process}}{\textit{tempo thread process}}$

Gestione degli errori

errno.h + string.h

Tipicamente, una system call POSIX restituisce -1 per segnalare il fallimento dell'operazione richiesta.

int errno (variabile globale):
- per poterla ispezionare: **#include <errno.h>**;
- in caso di errore, viene dettata con un opportuno codice che indica la causa del fallimento;
- è possibile ottenere una descrizione testuale del codice di errore tramite la funzione **sterror()** da **<string.h>**.

Esempio:

```
#include <errno.h>
#include <stdio.h>
#include <string.h>
...
int ret = my_call(arg1, ..., argN);
if (ret) {
        printf("Error in my_call: %s\n", strerror(errno));
        exit(EXIT_FAILURE);
}
```

perror(const char*str)

```
#include <stdio.h>
#include <errno.h>

void perror(const char *str);
```

- Stampa la stringa **str**, seguita dal messaggio di errore relativo ad errno.
- Se **str** è **NULL** stampa solo il messaggio di errore relativo ad **errno**.
- L'output è stampato sul default error stream.

Macro definiti in common.h

```
// for most common syscalls
#define handle_error(msg) \
do { \
        perror(msg); \
        exit(EXIT_FAILURE); \
} while (0)

// for phtread library only and other functions that do not set errno
#define handle_error_en(ret, msg) \
do { \
        errno = ret; \
        perror(msg); \
        exit(EXIT_FAILURE); \
} while (0)
```

Servono a gestire un errore in una riga sola, invece di riscrivere ogni volta il blocco con **printf** e **exit**.

- **handle_error(msg)**: per le system call più comuni, che in caso di errore dettano **errno.perror(msg)** stampa msg seguito dal messaggio di errore relativo a **errno**, poi **exit(EXIT_FAILURE)** termina il programma.
- **handle_errore_en(ret, msg)**: per la libreria pthread e le altre funzioni che non dettano **errno**, ma restituiscono direttamente il codice di errore. **errno = ret** copia quel codice in **errno**, cosi **perror** può stamparne il messaggio corretto.

Concorrenza in C

Semafori

Le Operazioni sul semaforo sono tre:

1. **Inizializzazione**: assegna un valore iniziale non negativo al semaforo.
2. **Operazione semWait**: decrementa il valore del semaforo; se il valore è negativo il processo/thread viene messo in attesa in una coda, altrimenti va avanti.
3. **Operazione semSignal**: incrementa il valore del semaforo; se il valore non è positivo un processo/thread viene risvegliato dalla coda.

Semafori POSIX

```
#include <semaphore.h>
…
sem_t sem;
…
sem_init(&sem, pshared, value)
…
sem_wait(&sem)
…
sem_post(&sem)
…
sem_destroy(&sem)
```

- **#include <semaphore.h>**: header da includere.
- **sem_t sem**; : dichiarazione di una variabile di tipo **sem_t,** che rappresenta il nostro semaforo.
- **sem_init(&sem, pshared, value)**: inizializzazione del semaforo con valore valore:
  - Se **pshared** vale 0, il semaforo viene condiviso tra i thread del processo.
  - Altrimenti, il semaforo viene condiviso tra processi, a patto che sia in una posizione di memoria condivisa.
  - In caso di successo, viene ritornato 0; in caso di errore, -1.
- **sem_wait(&sem)**: operazione **semWait** sul semaforo **sem**. In caso di successo, viene ritornato 0; in caso di errore, -1.
- **sem_post(&sem)**: operazione **semSignal** sul semaforo **sem**. In caso di successo, viene ritornato 0; in caso di errore, -1.
- **sem_destroy(&sem)**: distrugge il semaforo **sem**. In caso di successo, viene ritornato 0; in caso di errore -1.

INDICE

Thread in C **1**

Primitive 1

Processi **2**

Processi vs Thread 2

Misurazione performance 2

Tempi di lancio/terminazione 3

Gestione degli errori **3**

errno.h + string.h 3

perror(const char*str) 4

Macro definiti in common.h 4

Concorrenza in C **4**

Semafori 4

Semafori POSIX 5